

Trove — Frontend Engineer Assessment

Overview

Build an **Investment Portfolio Dashboard** (web application) using React. The app should have a simple login screen and a dashboard that displays a user's portfolio — net worth, allocation breakdown, account summaries, holdings, and transactions.

You are provided with:

1. A **wireframe** showing the expected layout and flow
2. A **JSON data file** containing mock portfolio data
3. A **color palette** extracted from our design system

Stack:

- React (Next.js, Vite, or CRA — your choice)
 - Any CSS approach (Tailwind, Styled Components, CSS Modules, plain CSS — your choice)
 - Any charting library for the allocation bar (Recharts, Chart.js, D3, etc.)
 - No UI component libraries (no Material UI, Ant Design, Chakra, etc.) — we want to see your CSS
-

What to build

1. Login screen

A simple sign-in form with email and password fields. No real authentication is needed — simulate a successful login on form submission (with basic validation) and navigate to the dashboard. Handle edge cases: empty fields, invalid email format, a loading state on the button.

2. Dashboard

After login, display a single dashboard view containing:

Net worth and allocation

- A net worth card showing the total portfolio value (computed from all holdings), a percentage change indicator, and a hide/show balance toggle
- An allocation bar — a horizontal stacked bar (not a pie/donut chart) showing distribution of portfolio value by sector, with a legend beneath it

Account list

- Below the allocation, display a list of accounts/categories derived from the data (group holdings by sector or account type). Each account card should show the category name, number of positions, and total value.

Holdings and transactions (tabbed)

- Two tabs side by side: Stocks and Orders
- Stocks tab:** A card-style list of holdings (not a table). Each card shows: ticker, company name, number of shares, current value, and gain/loss (amount and percentage). Include a search bar to filter by ticker or company name, and filter pills to filter by sector (All, Technology, Healthcare, Finance, etc.)
- Orders tab:** A list of recent transactions showing: type (Buy/Sell), asset name, shares, total amount, date, and status. Include filter pills (All / Buy / Sell). Visually distinguish transaction statuses (Completed, Pending, Failed).

Design specification

Use the provided wireframe as a visual reference — it is **not** a pixel-perfect spec. We want to see your ability to create a polished, well-spaced UI from a loose guide.

Color palette (Trove v3)

Token	Value	Usage
Primary (Trove Green)	#059A83	Buttons, active states, links
Primary light	#E0F5E1	Badges, subtle backgrounds
Success / positive	#10AE17	Positive returns, gains
Negative / loss	#BF221C	Negative returns, losses, errors
Background canvas	#FBFBFB	Page background
Card surface	#FFFFFF	Card backgrounds
Page background	#F5F1EE	Outer page background
Background default	#F2F6F6	Input backgrounds, neutral fills
Text default	#13342F	Headings, primary text
Text neutral	#687D7A	Labels, secondary text
Text disabled	#92A29F	Placeholders, muted text
Border	#DBDFDF	Card borders, dividers

Token	Value	Usage
Accent blue	 #00B6DF	Highlights, info states
Purple	 #7B79C9	Chart segments
Cream	 #F2C891	Chart segments
Dark blue	 #00323D	Dark backgrounds

Typography

- Use any clean sans-serif font (Inter, DM Sans, or system default — your choice)
- Net worth value: 26-28px, semi-bold
- Card values: 14px, medium weight
- Labels and captions: 11-12px, neutral/disabled color

Layout

- The app must be **responsive across all device sizes** — desktop, tablet, and mobile
- Card border-radius: 14-16px
- Use spacing and whitespace generously

Data handling

You are provided a `portfolio_data.json` file. **Do not import it directly into your components.**

Create a **service layer** that wraps the JSON data and simulates asynchronous API calls. Your components should consume data through this service layer with proper loading and error states. How you structure this is up to you — we want to see your approach.

Intentional data quirks

The provided JSON contains a few deliberate issues. How you handle them is part of the assessment:

1. **NVDA has a `currentPrice` of `0`** — how do you display this and handle it in calculations?
2. **DIS has `0` shares** — is this a real holding? How does it affect the summary and allocation?
3. **A transaction has status `"PENDING"`** — should it look different from completed ones?
4. **A transaction has status `"FAILED"`** — same question.

5. **Gain/loss for negative returns** — proper formatting, color, and signage.

There are no "correct" answers — we want to see that you noticed these and made intentional decisions. Document your decisions in your README.

What to submit

1. A **GitHub or GitLab repository** (public or invite-shared) containing:
 - The complete source code
 - A `README.md` that includes:
 - How to run the project locally
 - Your approach and any architectural decisions
 - How you handled the data quirks above
 - What you would improve or add with more time
 2. A **deployed link** (Vercel, Netlify, or any free hosting)
-

Important notes

- **No backend is required.** The JSON file is your data source.
 - **No UI libraries.** We need to see your CSS skills. Charting libraries are fine.
 - **The wireframe is a guide, not a rigid spec.** Use your judgment to improve on it.
-

Good luck! If you have questions about the task (not the solution), reply to this email.